

TurtleGrow היא מערכת בענן שעוזרת לחקלאי לגדל בזיליקום: היא מזהה מחלות מתמונה, קוראת חיישנים מהשדה, נותנת המלצות חכמות, ואף משקה לבד דרך סוכן אוטונומי. המערכת דורשת הרשמה/התחברות, ושומרת לכל משתמש נקודות והיסטוריה. כל הפרויקט יושב בתא אחד של Google Colab `w3_turtles.ipynb`), הבנוי בארכיטקטורת **מיקרו-שירותים**.

1. ארכיטקטורה כללית

התא הראשי בונה תחילה שלושה קבצי-שירות נפרדים, מייבא אותם בחזרה, ומחבר את הכל למסך אחד עם 6 טאבים + מסך התחברות + סוכן אוטונומי:

```
TurtleGrow Notebook (UI) - ipywidgets, 6 Tabs + Auth + Agent
|
+-- Gemini LLM          -> RAG engine: writes the expert answers
+-- Firebase Realtime DB -> cloud database: users, points, index, history
+-- IoT Server (Render) -> live sensor readings
+-- HuggingFace Hub Vision -> disease-from-image model (talzablev/final-basil-model)
+-- Vimeo              -> irrigation video (auto-closing pop-up)
```

מי עושה מה:

שירות	מה הוא נותן למערכת	איך מתחברים אליו
Google Gemini	"המוח" — כותב תשובות מקצועיות בשפה טבעית	ספריית <code>google-generativeai</code>
Firebase Realtime DB	בסיס הנתונים בענן — משתמשים, סיסמאות, נקודות, אינדקס, היסטוריה	קריאות HTTP (REST)
שרת IoT (ב-Render)	קריאות חיישנים אמיתיות (טמפ'/לחות/קרקע)	בקשות HTTP עם <code>requests</code>

שירות	מה הוא נותן למערכת	איך מתחברים אליו
מודל Hugging Face Hub	"העיניים" — מזהה מחלה מתמונה	<code>transformers.pipeline</code> (מוריד מה-Hub)
Colab Secrets	מקום מאובטח למפתח ה-API	<code>userdata.get</code>

2. קבצים ומבנה

הפרויקט הוא תא אחד (`w3_turtles.ipynb`) שיוצר בזמן ריצה תיקיית שירותים. עיקרון מרכזי: **הפרדה למיקרו-שירותים** — כל תחום אחריות בקובץ משלו:

קובץ	מה יש בו
<code>w3_turtles.ipynb</code>	התא הראשי: הגדרות, בניית השירותים, המסכים, וההרצה
<code>services/iot_service.py</code>	<code>IoTService</code> — קריאת חיישנים + מדד אמינות
<code>services/db_service.py</code>	<code>DatabaseService</code> — פעולות מול Firebase (get/post/put/delete)
<code>services/search_services.py</code>	חמש מחלקות ה- <code>SearchOrchestrator</code> + RAG שמחבר ביניהן
<code>system_debug.log</code>	קובץ לוג נסתר למפתח (שגיאות + מדדי KPI)

מבנה הנתונים ב-Firebase:

- `/users/{שם-משתמש}` — סיסמה ונקודות של כל משתמש רשום.
- `/search_engine/inverted_index` — האינדקס ההפוך השמור בענן.
- `/rag_search_results` — היסטוריית שאלות ותשובות (עם שם המשתמש).
- `/iot_sensor_data` — גיבוי קריאות חיישנים.
- `/agent_actions` — יומן החלטות הסוכן האוטונומי.

3. אימות משתמשים — התחברות, הרשמה ואורח

מסך הבית (מסך 0) חוסם את המערכת עד שהמשתמש מזדהה. שלוש דרכים: **התחברות**, **הרשמה**, או **כניסה כאורח** (בלי שמירת נתונים). המשתמש הפעיל נשמר במשתנה גלובלי `CURRENT_USER`.

הרשמה — בודקים ששם המשתמש פנוי, ואם כן יוצרים רשומה חדשה ב-Firebase:

```
def handle_register(b):
    user, pwd = username_input.value.strip(), password_input.value
    # בודקים אם שם המשתמש כבר תפוס
    existing_user = db_service.get_data('/users', user)
    if existing_user:
        print("❌ Username already taken.")
    else:
        # יוצרים משתמש חדש עם אפס נקודות
        db_service.put_data('/users', user, {'password': pwd, 'points': 0})
        global CURRENT_USER
        CURRENT_USER = user
```

התחברות — שולפים את המשתמש ומשווים סיסמה:

```
def handle_login(b):
    user, pwd = username_input.value.strip(), password_input.value
    # שולפים את רשומת המשתמש מהענן
    user_data = db_service.get_data('/users', user)
    if user_data:
        # משווים את הסיסמה שהוזנה לזו השמורה
        if user_data.get('password') == pwd:
            global CURRENT_USER
            CURRENT_USER = user # המשתמש מחובר
        else:
            print("❌ Incorrect password.")
    else:
        print("❌ Username not found.")
```

אורח / התנתקות — אורח מקבל את השם "Anonymous_Farmer" ונחסם ממסך המשחק;
`handle_logout` מאפס את `CURRENT_USER` ומחזיר את מסך ההתחברות.

הערה למפתח: הסיסמאות נשמרות ב-Firebase כטקסט גלוי (ללא הצפנה/hash) — מתאים לפרויקט לימודי, לא לסביבת ייצור.

4. שכבת המיקרו-שירותים

התא הראשי כותב את קוד השירותים למחרוזות, שומר אותן כקבצי `.py`, ואז מייבא אותן — כך שהלוגיקה מופרדת ומאורגנת כמו בפרויקט אמיתי:

```
# כותבים את קוד השירות לקובץ
with open('services/iot_service.py', 'w', encoding='utf-8') as f:
    f.write(iot_service_content)
# מייבאים אותו בחזרה כמודול רגיל
from services.iot_service import IoTService
```

IoTService — קריאת חיישנים + מדד אמינות

שולף טמפל/לחות/קרקע, וסופר כמה בקשות הצליחו כדי לחשב **אחוז אמינות** (KPI) שנרשם ללוג:

```
def get_latest_sensor_data(self):
    self.total_requests += 1
    try:
        # שולפים את הקריאה האחרונה של כל חיישן
        # ...
        # מחשבים את אחוז ההצלחה ורושמים אותו ללוג
        success_rate = ((self.total_requests - self.failed_requests) / self.total_requests)
    except Exception:
        # כישלון מגדיל את מונה הכשלים ומוריד את אחוז האמינות
        self.failed_requests += 1
```

מרכז את כל הפעולות מול בסיס הנתונים (get_data, post_data, put_data, delete_data). כל פעולה עוטפת בקשת HTTP ומחזירה ערך ריק בשקט אם משהו נכשל — כדי לא לקרוס.

שירותי החיפוש (RAG) — חמש מחלקות + מתזמן

במקום מחלקה אחת גדולה, ה-RAG פוצל לפי אחריות:

- `UserIdentityService` — מזהה את המשתמש (או "אורח").
- `IndexBuilderService` — בונה/טוען את האינדקס ההפוך (הסבר בהמשך).
- `QueryHandlerService` — מדרג את המאמרים לפי מילים משותפות עם השאלה.
- `ResultProcessorService` — שולח ל-Gemini, שומר את התוצאה ומחזיר מקורות.
- `SearchOrchestrator` — **המתזמן**: מחבר את ארבעת השירותים לפעולה אחת פשוטה.

```
class SearchOrchestrator:
    def ask_expert(self, query_string, username_input=None):
        # שלב ראשון - מזהים את המשתמש
        user_id = self.user_service.identify_user(username_input)
        # שלב שני - מאתרים מאמרים רלוונטיים
        relevant_docs = self.query_service.retrieve_documents(query_string)
        # שלב שלישי - מייצרים תשובה, שומרים, ומחזירים מקורות
        answer, sources = self.result_service.generate_and_save(query_string, relevant_docs)
        return answer, sources
```

מהו RAG ואיך בונים אינדקס הפוך? RAG = קודם **מאתרים** קטעי טקסט רלוונטיים מתוך 5 תקצירים אקדמיים, ואז **נותנים אותם ל-Gemini שיכתוב** תשובה רק על סמך מה שנמצא (מונע "המצאות"). האינדקס ההפוך הוא טבלה שאומרת, לכל מילה, באילו מאמרים היא מופיעה. לפני ההכנסה מנקים כל מילה — מורידים מילים נפוצות ומצמצמים לשורש (Porter Stemmer):

```
for word in words:
    # מדלגים על מילים נפוצות וחסרות-משמעות
    if word not in self.stop_words:
        # מצמצמים את המילה לשורש שלה
```

```
stemmed_word = self.stemmer.stem(word)

# מוסיפים את מספר המאמר לרשימת המאמרים של אותו שורש
raw_index_data[stemmed_word].add(doc_id)
```

האינדקס נבנה פעם אחת ונשמר ב-Firebase; בריצות הבאות טוענים אותו מוכן (cloud caching).

5. הסוכן האוטונומי — AutonomousCropAgent

סוכן שמחליט לבד מתי להשקות, לפי לולאה קלאסית של **חשיבה** → **פעולה** (sense-think-act):

- **sense** — קורא טמפרטורה ולחות-קרקע מהשרת.
- **think** — מחליט לפי ספים: אם הקרקע יבשה **וגם** חם — מצב **TRIGGER**; אם רק יבשה — **WARNING**;
- אחרת — **IDLE**.
- **act** — במצב **TRIGGER** מפעיל השקיה ומתעד את ההחלטה ב-Firebase.

```
def think(self, temp, soil):
    # אם אין נתונים - לא מחליטים כלום
    if temp is None or soil is None:
        return "IDLE", "Awaiting sensor data..."
    # קרקע יבשה וגם חום גבוה = דחוף להשקות
    if soil < self.CRITICAL_SOIL_MOISTURE and temp > self.HIGH_TEMP_THRESHOLD:
        return "TRIGGER", f"Critical high-heat evaporation! Soil {soil}%, Temp {temp}C."
    # רק קרקע יבשה = אזהרה בלבד
    elif soil < self.CRITICAL_SOIL_MOISTURE:
        return "WARNING", f"Low soil moisture ({soil}%)."
    # אחרת - הכל תקין
    else:
        return "IDLE", "Conditions optimal."
```

6. מסכי ה-UI — איך כל מסך עובד

מסך 1 — אבחון חכם (on_image_upload): השרשרת המרכזית

event-handler שרץ בכל העלאת תמונה, מחבר את שלושת השירותים, מודד את זמן התגובה (KPI), ומעניק נקודות למשתמש רשום:

```
#שלב ראשון - העיניים: מה רואים בתמונה?
hf_disease = analyze_image_locally(content)

#שלב שני - החיידנים: מה מצב השדה?
current_temp, current_hum, current_soil = get_latest_sensor_data()

#שלב שלישי - המוח: המודל כותב את הדוח
response = diagnostic_model.generate_content(rag_prompt)

#מודדים את זמן התגובה ורושמים אותו כמדד ביצועים
latency = round(time.time() - start_time, 2)

#מעניקים עשרים נקודות למשתמש רשום (לא לאורח)
if CURRENT_USER and CURRENT_USER != "Anonymous_Farmer":
    db_service.put_data(f'/users/{CURRENT_USER}', 'points', current_user_pts + 20)
```

מסך 2 — חיישני (fetch_iot_data) IoT: נתונים + השקיה אוטומטית

מושך היסטוריה דרך `IoTService`, מסדר בטבלה עם יחידות, ואם הקרקע יבשה מפעיל השקיה:

```
#אם זה חיישן הקרקע
if feed == "soil":
    #שולפים את המספר מתוך הערך המעוצב
    latest_soil_val = float(str(df["value"].iloc[0]).split())[0])

    #אם הקרקע יבשה מדי - מפעילים השקיה
    if latest_soil_val < 40.0:
        trigger_irrigation_event(...)
```

מסך 3 — יועץ אגרונום (perform_search): שאלה + מקורות

קורא ל- `SearchOrchestrator` ומציג את התשובה יחד עם **קישורים לחיצים למקורות** — לב השקיפות:

```
# קריאה אחת פשוטה למתזמן – הוא עושה את כל העבודה
answer, sources = search_engine.ask_expert(q, active_user)
# מציגים כל מקור כקישור לחיץ
for s in sources:
    html_content += f"<li><a href='{s['url']}'>{s['title']}</a></li>"
```

מסך 4 — דאשבורד (draw_dashboard): שני גרפים + קו מגמה

מציג כרטיסי מצב ו-**שני** גרפים זה לצד זה: היסטוריית טמפרטורה, וגרף פיזור טמפרטורה-מול-לחות עם **קו מגמה** שמחושב בעזרת `numpy`:

```
# מתאימים קו מגמה ישר לנקודות (רגרסיה לינארית פשוטה)
z = np.polyfit(df_merged['temp'], df_merged['soil'], 1)
p = np.poly1d(z)
# מציירים את קו המגמה מעל נקודות הפיזור
ax2.plot(df_merged['temp'], p(df_merged['temp']), "r--", label='ML Trendline')
```

מסך 5 — משחוק (complete_task / sync_user_data): נקודות לפי משתמש

המשחוק תלוי בהתחברות: **אורח נחסם** ורואה מסך נעילה; משתמש רשום רואה ניקוד, תג (badge), פס-התקדמות ולוח מובילים. השלמת משימה מוסיפה נקודות ושומרת אותן בענן תחת המשתמש:

```
def complete_task(b, points):
    global current_score
    # מוסיפים את הנקודות
    current_score += points
    # במשימת ההשקיה מפעילים גם את הסרטון
    if points == 50:
        trigger_irrigation_event("Completed Daily Task: Water morning plants")
    # שומרים את הניקוד בענן תחת המשתמש הפעיל
    db_service.put_data(f'/users/{active_user}', 'points', current_score)
```

7. מנוע האוטומציה — פופ-אפ שנסגר לבד

`trigger_irrigation_event` פותח חלון קופץ עם התראה + סרטון השקיה, ומשתמש ב-`asyncio` כדי לסגור אותו אוטומטית אחרי 12 שניות — בלי להקפיא את הממשק:

```
# משימת רקע שסוגרת את החלון לבד אחרי המתנה קצרה
async def auto_close():
    await asyncio.sleep(12)
    video_modal.layout.display = 'none'
# מפעילים אותה ברקע בלי לחסום את הממשק
asyncio.ensure_future(auto_close())
```

8. תבניות עיצוב ורעיונות מעניינים

א. **מיקרו-שירותים** — הקוד מפוצל לקבצי-שירות עצמאיים (IoT, DB, Search) הנכתבים בזמן ריצה ומיובאים בחזרה. כל שירות עצמאי ובדיק בנפרד.

ב. **מתזמן (Orchestrator / Facade)** — `SearchOrchestrator` מסתיר את מורכבות ארבעת שירותי ה-RAG מאחורי פונקציה אחת פשוטה (`ask_expert`).

ג. **סוכן אוטונומי** — לולאת `sense-think-act` שמקבלת החלטות לבד ומתעדת אותן, בלי מעורבות משתמש.

ד. **מדדי KPI בלוג** — המערכת מודדת את עצמה: אחוז אמינות ה-IoT, זמן תגובת ה-AI, ומעורבות המשתמשים — הכל נרשם ל-`system_debug.log`.

ה. **צבעים במקום אחד** — כל הצבעים מוגדרים פעם אחת כמשתנים (`COLOR_PRIMARY` וכו') ומשמשים בכל המסכים.

ו. **שגיאות מוסתרות מהמשתמש** — "יומן" נסתר כותב את כל השגיאות לקובץ; המשתמש רואה ממשק נקי בלבד.

ז. **"כישלון שקט"** — פעולות מול הענן עטופות כך שכשל רשת רק נרשם ללוג ולא הורס את חוויית המשתמש.

ח. **התמודדות עם שרת איטי (Cold-start)** — כל בקשה מחכה עד 60 שניות כי השרת החינמי ב-Render "נרדם", עם הודעה מרגיעה למשתמש.

ט. **אפליקציה במסך אחד + ניתוב טאבים** — כל מסך נארז בנפרד ומוכנס לרכיב טאבים; מעבר לטאב המשחק מרענן את נתוני המשתמש:

```
# כשעוברים לטאב המשחק (מספר חמש) מרעננים את נתוני המשתמש
def on_tab_change(change):
    if change.new == 5:
        sync_user_data()
```

9. זרימת הנתונים — אבחון תמונה (השרשרת המרכזית)

```
[1] Image upload
    |
    v
analyze_image_locally ..... HuggingFace Vision --> disease label
    |
    | [2] get_latest_sensor_data .. IoT / Render --> temp, humidity, soil
    |     |
    v     v
+-----+
| Build combined prompt |
+-----+
    |
    v
Gemini LLM --> report + award points to the logged-in user
```

במילים פשוטות: התמונה הולכת למודל הזיהוי ומחזירה שם מחלה; במקביל נשלפים נתוני החיילים; שתי התוצאות מתחברות להוראה אחת ל-Gemini; הדוח מוצג, ולמשתמש רשום מתווספות נקודות.

10. תלויות (ספריות שהפרויקט צריך)

```
google-generativeai · pandas · numpy · matplotlib · plotly · requests  
PyPDF2 · nltk · ipywidgets · pillow · transformers · asyncio
```

בנוסף — סביבת `google.colab.userdata` Google Colab למפתח הסודי).